



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 09-04-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.-Agregar movimiento con teclado al helicóptero hacia adelante y atrás.

Para el ejercicio se implementó el movimiento del helicóptero utilizando el teclado, para lograr que el helicóptero tuviera un movimiento independiente del coche se replicó la lógica utilizada previamente en la variable muevex pero se creó una nueva variable llamada muevex2 para el helicóptero, de esta manera el desplazamiento del helicóptero se calcula de forma independiente dentro del main, lo que permitió controlar el movimiento del helicóptero sin afectar el movimiento del coche.

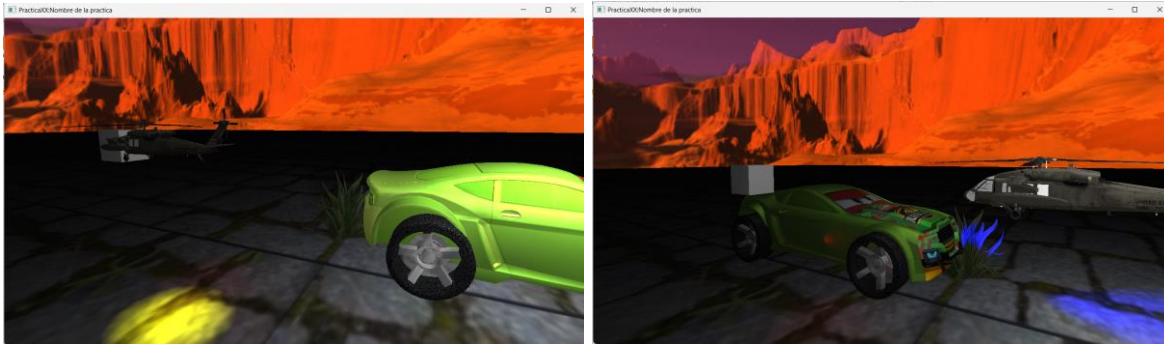
```
16 GLfloat getmuevex() { return muevex; }  
17 GLfloat getmuevex2() { return muevex2; }
```

```
43 GLfloat muevex;  
44 GLfloat muevex2;
```

```
28 muevex = 2.0f;  
29 muevex2 = 0.0f;
```

```
206 // HELICÓPTERO ATRÁS  
207 if (key == GLFW_KEY_J)  
208 {  
209     theWindow->muevex2 += 0.5f;  
210 }  
211  
212 // HELICÓPTERO ADELANTE  
213 if (key == GLFW_KEY_K)  
214 {  
215     theWindow->muevex2 -= 0.5f;  
216 }  
217
```





2.- crear luz spotlight de helicóptero de color amarilla que apunte hacia el piso y se mueva con el helicóptero

Se implementó una luz tipo spotlight de color amarillo la cual simula un reflector que apunta hacia el suelo desde el helicóptero, primero se creó la luz utilizando la clase `SpotLight` definiendo su color intensidad, dirección y parámetros de atenuación, posteriormente para que la luz siguiera al helicóptero, se aplicó el concepto de jerarquía de transformaciones, similar al utilizado en los modelos del coche, para esto se guardó la matriz del helicóptero como nodo padre, Después, se creó una matriz hija para la luz aplicando un offset respecto al helicóptero, finalmente, se obtuvo la posición de la luz a partir de la matriz y se actualizó el spotlight, así se logró que la luz se mueva de manera coherente con el helicóptero cumpliendo con el requisito de jerarquía.

```

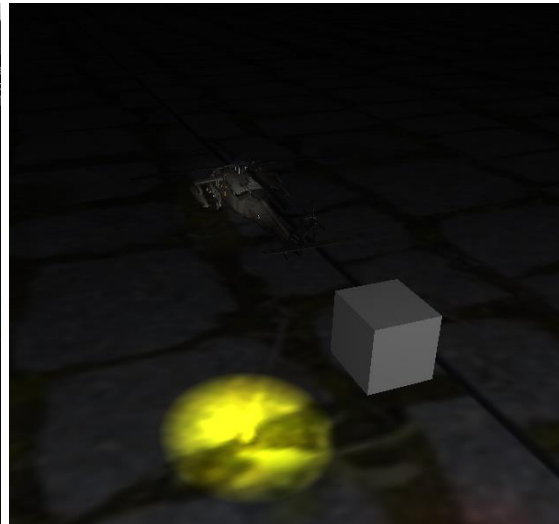
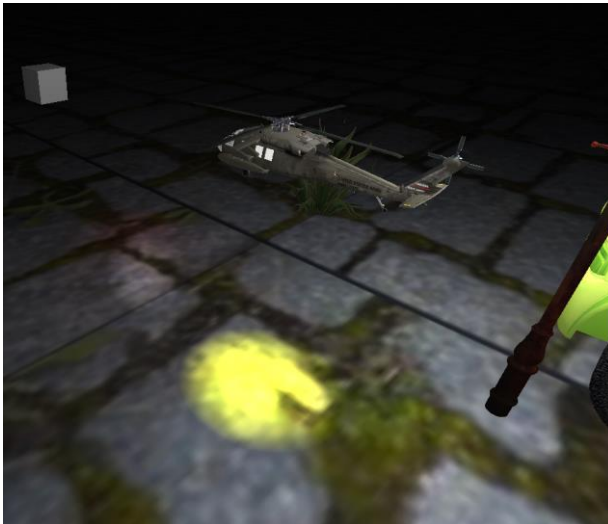
438 | //----- luz helicoptero -----
439 | spotLights[3] = SpotLight(
440 |     1.0f, 1.0f, 0.0f,    // color amarillo
441 |     0.5f, 0.8f,         // intensidad
442 |     0.0f, 0.0f, 0.0f,   // posición inicial
443 |     0.0f, -5.0f, 0.0f,  // hacia el piso
444 |     0.1f, 0.05f, 0.01f, // atenuación (alcance medio)
445 |     20.0f               // ángulo (cono)
446 | );
447 | spotLightCount++;
448 | //-----
449 |

```

```

611
612 //----- helicoptero -----
613 model = glm::mat4(1.0);
614 model = glm::translate(model, glm::vec3(
615     mainWindow.getmuevex2(),
616     5.0f,
617     6.0f
618 ));
619 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
620 model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
621 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
622
623 //guardar como padre
624 glm::mat4 modelHeli = model;
625
626 // render
627 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
628 Blackhawk_M.RenderModel();
629
630
631 // matriz hija
632 glm::mat4 modelLuzHeli = modelHeli;
633
634 // offset (debajo del helicóptero)
635 modelLuzHeli = glm::translate(modelLuzHeli, glm::vec3(0.0f, -2.0f, 0.0f));
636
637 glm::vec3 posHeli = glm::vec3(modelLuzHeli[3]);
638 glm::vec3 dirHeli = glm::vec3(0.0f, -1.0f, 0.0f);
639
640 // actualizar spotlight
641 spotLights[3].SetFlash(posHeli, dirHeli);
642
643 //-----

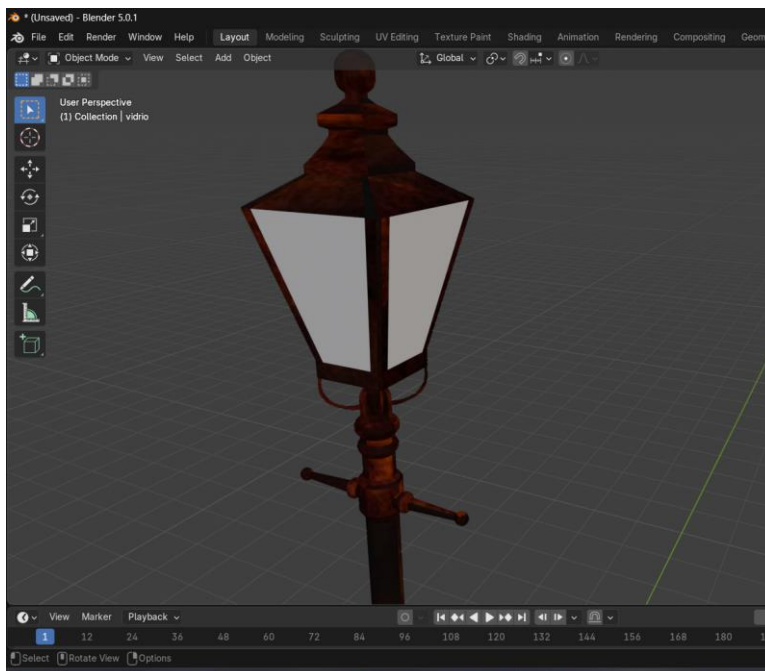
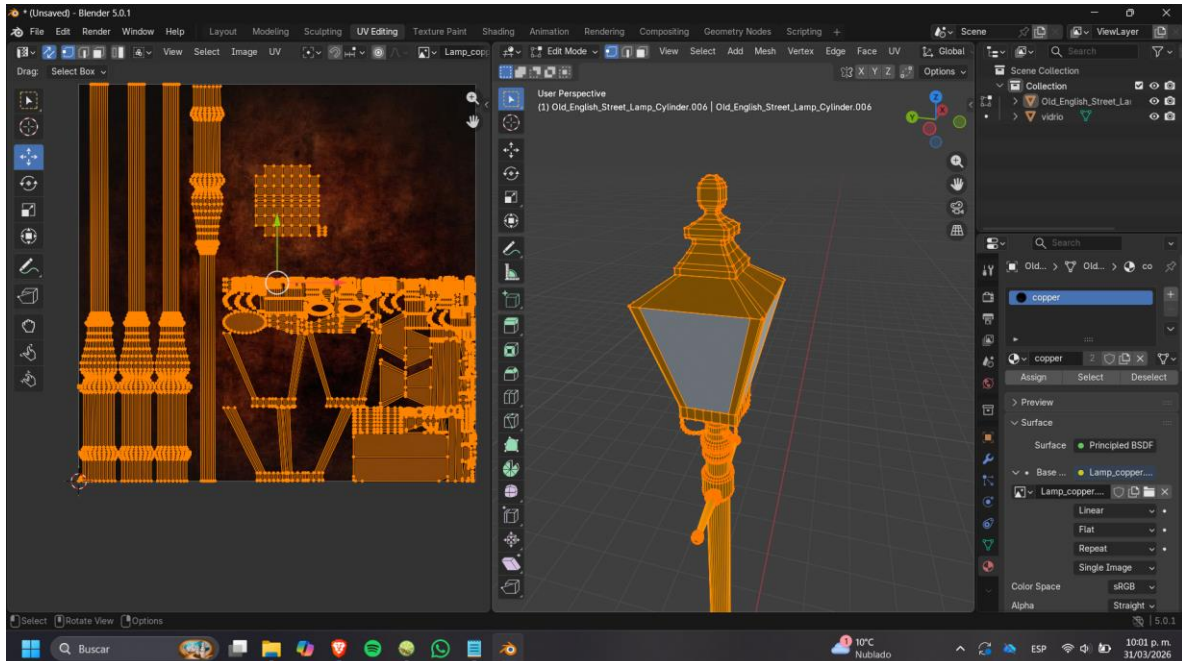
```



3.- Añadir en el escenario 1 modelo de lámpara texturizada y crearle luz puntual blanca

En este ejercicio se agregó un modelo de lámpara de estilo victoriano para que coincidiera con el estilo de tecnología a vapor/steampunk del proyecto al escenario, el texturizado del modelo se realizó previamente en blender, donde se asignaron los materiales correspondientes para darle una textura de cobre oxidado y vidrio blanco, posteriormente se implementó una luz de tipo point light para simular el foco de la lámpara, la luz fue configurada con un color blanco y

parámetros de atenuación para limitar su alcance, para que la luz estuviera correctamente posicionada respecto a la lámpara se utilizó un translate para colocarla en las mismas coordenadas del modelo de la lampara aumentando la altura (eje y) para que correspondiera a la posición de la parte de cristal del farol.




```

96     Model Carro_LLanta4;
97     Model lampara;
98

```

```

357
358     lampara = Model();
359     lampara.LoadModel("Models/lampara.obj");
360

```

```

393     //----- foco lampara -----
394     pointLights[1] = PointLight(
395         1.0f, 1.0f, 1.0f, // color blanco
396         0.5f, 0.5f,      // intensidad ambiental y difusa
397
398         9.0f, 7.5f, 3.0f, // posición
399
400         0.1f, 0.05f, 0.01f // atenuación (alcance medio) 0.05f, 0.02f, 0.005f,
401     );
402     pointLightCount++;
403     //-----
404
405     unsigned int pointLightCount = 0;

```

```

644     //----- lampara -----
645     model = glm::mat4(1.0);
646     model = glm::translate(model, glm::vec3(9.0f, -0.5f, 3.0f));
647
648     // guardar como padre
649     glm::mat4 modelLampara = model;
650
651     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
652     lampara.RenderModel();
653
654
655     //-----
656

```



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Inicialmente se intentó posicionar las luces sumando coordenadas manualmente, lo cual no representaba una jerarquía real, esto se resolvió utilizando matrices (model y modelaux) para heredar transformaciones.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
los ejercicios implicaron un nivel medio de complejidad ya que no solo se trató de crear luces sino de integrarlas correctamente dentro de la estructura jerárquica del programa.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
La explicación en clase fue suficiente para comprender como funcionan los distintos valores que afectan las fuentes de iluminación creadas, solo sería conveniente llevar un ritmo más lento debido a que no todos los equipos son capaces de aplicar y volver a compilar los cambios tan rápido
- c. Conclusión
En esta práctica se trabajó con distintos tipos de iluminación en OpenGL, aplicando conceptos de transformación y jerarquía para lograr que las luces se comportaran de manera coherente con los objetos de la escena, la práctica nos permitió comprender mejor cómo se manejan las transformaciones y cómo estas pueden aplicarse también a sistemas de iluminación. Como área de mejora, sería conveniente profundizar más en la implementación interna de las clases de luz para permitir mayor flexibilidad, como el uso de métodos para actualizar propiedades dinámicamente.

1. Bibliografía en formato APA

- Learn OpenGL. (s.f.). Lighting. Recuperado de: <https://learnopengl.com/Lighting>
- OpenGL Documentation. (s.f.). Recuperado de: <https://www.opengl.org/documentation/>
- Khronos Group. (s.f.). OpenGL Reference Pages. Recuperado de: <https://registry.khronos.org/OpenGL/>